TOR Front-End Roadmap

FastAPI + Jinja2 + HTML/CSS/JS: a focused, step-by-step learning plan for the TOR leaderboard and player cards.

Project goal

A professional-looking TOR dashboard: a full-width ranked leaderboard (narrow Rank and number columns, flexible Name column), where clicking a player pops a card over the page with their headshot, raw data, component breakdown, and a date-range histogram.

Starting point

The scoring engine and calibration config already exist and stay untouched. A `server.py` has been generated but is **not yet understood**, and that comes first. This roadmap covers only the web layer: serving data, templates, markup, styling, and interaction.

Ground rules

- Read only the doc section for the current step. Attempt a minimal version yourself. Then test it in the browser before adding anything.
- One change at a time. Commit to Git after each milestone.
- Verified means you saw it work in the browser, not that code was generated.
- No CSS frameworks (Tailwind, Bootstrap) and no JS frameworks until plain versions are understood.

Step 0. Understand server.py

Priority: First, before any HTML.

Docs: FastAPI tutorial: First Steps, Path Parameters, and Query Parameters sections.

Focus:

- What the `app` object, a route decorator, and a path function each do.
- How uvicorn runs the app, and what happens between typing a URL and getting a response.
- Where `server.py` calls the TOR engine, and what shape of data comes back.
- Which routes exist and what each one returns.

Milestone: Run the server, open each route in the browser, and explain in your own words what every route does and where its data comes from.

Note: If any line in `server.py` is unclear, mark it and ask. Do not build on code you cannot explain.

Step 1. Render data with Jinja2

Priority: Second.

Docs: FastAPI: Templates. Jinja2: Template Designer Documentation (variables, `for` loops, filters).

Focus:

- How a route passes data into a template.
- Looping over leaderboard rows and printing fields.
- Formatting numbers with filters (for example, one decimal place on the score).
- Template inheritance (a base layout), but only once one page works.

Milestone: A route renders a plain, unstyled HTML page showing the real top 25 players from the engine.

Step 2. HTML structure

Priority: Third.

Docs: MDN HTML basics, plus the pages for table elements and `data-*` attributes.

Focus:

- Page skeleton: head, body, header, main.
- Leaderboard markup (a semantic table or a grid of rows).
- Attaching a stable player ID to every row with a `data-` attribute. Never identify a player by name alone.

Milestone: Clean, valid markup where each row carries a player ID you can read back later.

Step 3. CSS layout and visual design

Priority: Fourth, and the step that fixes the Streamlit table problem.

Docs: MDN CSS basics, CSS Grid guide, box model, custom properties (CSS variables).

Focus:

- Grid column definitions: fixed widths for Rank and numbers, `1fr` for Name.
- Spacing, typography, and a dark theme defined with CSS variables.
- Row hover and selected states.
- Responsive behavior at narrower widths.

Milestone: The leaderboard spans the full page width with narrow numeric columns and a flexible Name column, and looks sharp enough that you would show it in a portfolio.

Note: Define colors and sizes once as variables so later restyling is a one-file edit.

Step 4. The player card as a native dialog

Priority: Fifth.

Docs: MDN: the `<dialog>` element, `showModal()`, `close()`, and the `::backdrop` pseudo-element.

Focus:

- A modal that pops over the leaderboard while the page underneath stays unchanged.
- Opening and closing it, including the Escape key and clicking outside.
- Styling the card layout: image top-left, raw data bottom-left, histogram top-right, component breakdown bottom-right (leading idea).

Milestone: Clicking a row opens a placeholder card over the page that shows that row's player ID, and it closes cleanly.

Step 5. JavaScript, only what is needed

Priority: Sixth.

Docs: MDN JavaScript Guide: only the sections on selecting DOM elements, events, `fetch`, and async/await.

Focus:

- Listening for row clicks and reading the player ID from the row.
- Calling a new FastAPI route such as `/player/{id}` with `fetch`.
- Filling the card with the JSON that comes back.
- Handling a failed or empty response.

Milestone: Clicking a row fetches that player's data from FastAPI and displays their name, TOR score, and component scores in the card.

Step 6. Headshots and card content

Priority: Seventh.

Focus:

- Decide where the headshot image comes from before writing display code.
- The engine must keep the batter ID to make that mapping possible.
- A fallback image when no headshot exists.
- Component breakdown display.

Milestone: The right headshot appears on every card, and a missing one fails gracefully.

Step 7. Histogram and date range

Priority: Eighth.

Docs: Chosen after Step 5. Likely Chart.js or Plotly.js loaded by script tag, plus MDN date input.

Focus:

- The engine must keep `game_date` on each at-bat.
- Start with one histogram and one variable.
- Date inputs that call a FastAPI route with start and end dates and redraw the chart.
- Handling invalid ranges and empty results.

Milestone: Choosing a date range updates the card's histogram to show only that window.

Rule: Filtering chart observations must never silently recalculate the official TOR score.

Step 8. Performance and polish

Priority: Last.

Docs: FastAPI: Static Files. Python `functools.lru_cache`, or loading data once at server startup.

Focus:

- Serving CSS, JS, and images as static files.
- Loading the engine's results once instead of on every request (the cold load was about 19 seconds).
- Loading, empty, and error states, a page title, a favicon.

Milestone: The page loads fast, and every failure mode shows a clear message instead of a blank space.

Recommended sequence

Step	Topic	First milestone
0	Read server.py	Explain every route
1	Jinja2 templates	Real data on a plain page
2	HTML structure	Rows carry player IDs
3	CSS grid	Sharp full-width leaderboard
4	`<dialog>`	Click opens a placeholder card
5	JavaScript + fetch	Card shows real player data
6	Headshots	Correct image per player
7	Histogram + dates	Date range updates chart

8	Static files + caching	Fast, polished, handles errors

Bookmarks folder (most used first)

1. HTML (MDN)
2. CSS (MDN)
3. Jinja2
4. FastAPI
5. JavaScript (MDN)

Your first milestone

Run `server.py`, hit every route in the browser, and explain what each one does. Then render the real leaderboard on a plain HTML page. Styling comes after that.